University of Bedfordshire

Principles of Programming and Data Structures

CIS096-1

Gym Membership Management System

Student Name: Hamzeh Abdo

Student Number: 702510039

Submission Date: 03/04/26

1. Introduction

This gym membership management system is designed to assist gyms and fitness centres to manage their memberships efficiently and accurately. This system uses object oriented software to help many small and medium sized centres to move away from relying on spreadsheets or manual records. Using spreadsheets or manual records can result in data inconsistencies, errors in calculating fees, and makes it harder to track if a membership is active or expired. This system is designed to solve all these issues along with helping with scalability and making it easier to manage information in general.

This system allows gym staff members to register members, create new membership plans, assign memberships, update member information, calculate fees, check status of memberships, and view the general details of the membership. This design follows the use case and class diagrams that were previously designed to maintain consistency with the previous reports.

2. System Architecture

This system is based on an object oriented design. The application is divided into more than one logical component. Each class in this system has its specific role and can interact with other classes if needed.

Presentation Layer

By using Java Swing components such as JFrame, JButton, and JOptionPane. This layer represents the graphical user interface which the gym staff can interact with through the buttons on the application.

Application Logic

This is the layer that holds the core functionality of the gym system. Operations like registering members, creating membership plans, and assigning memberships are all managed by this layer.

These are the classes responsible for this logic:

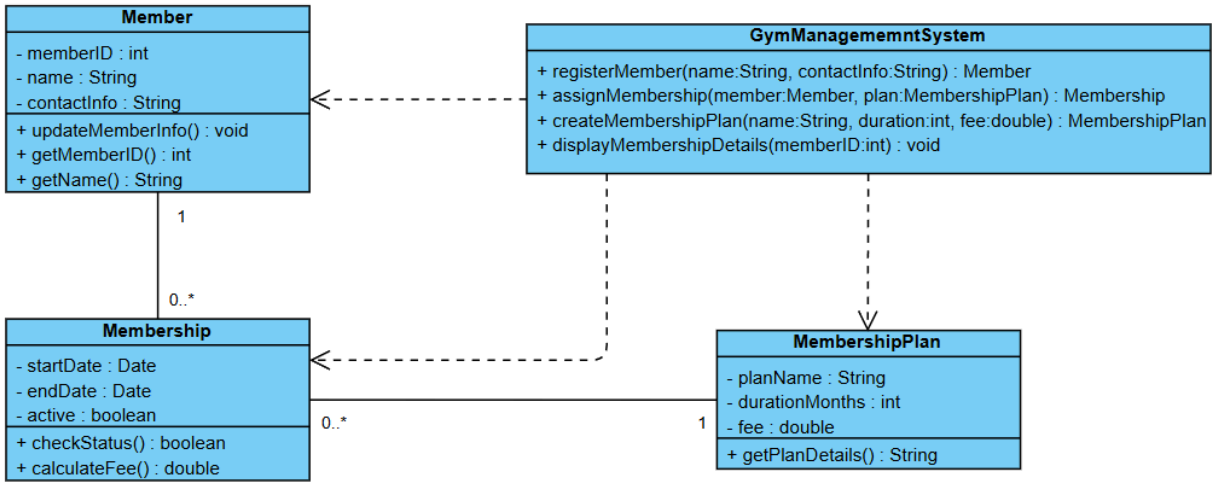
- GymManagementSystem
- Member
- Membership
- MembershipPlan

Data Layer

This stores the objects while the program is executed. Data is stored by using ArrayList that holds a collection of members, and membership plans.

3 UML Diagrams

3.1 Class Diagram



The system consists of four main classes including Member, MembershipPlan, Membership, and GymManagementSystem.

GymManagementSystem: The central controller that coordinates almost all interactions between the members and the membership plans. It is mainly responsible for registering new members, creating the new membership plans, assigning memberships to the members, and displaying member details.

Member: This class represents a gym member. It stores details like their ID, name, and contact information. This class interacts with the membership class when a plan is assigned.

MembershipPlan: This is the membership plan that shows information like the length of the plan, the type of the plan, and the fee. This would be assigned to a member through a membership.

Membership: This represents the membership that is assigned to a member. This class tracks start and end dates to make sure whether the plan is expired or not. It also calculates fees.

Diagram Description

The uml diagram shows the 4 classes and how they are connected and interact. The gym management system class shows a dependency to the other three classes which include, member, membership, and membership plan. There is also a one to many association from membership plan to membership, and from membership to member.

Member Class

The purpose of this class is to represent a gym member and verify their personal information.

Attributes

- memberID: int - unique way to identify each member
- Name: String - member name
- contactInfo: String - contact information like phone number and email

Methods

- getMemberId() - returns a member ID
- getName() - returns the name of the member
- updateMemberInfo() - updates the member's contact information
- getContactInfo() - returns contact information

Interactions

- When the GymManagementSystem.registerMember() is called, a new member object is created
- The member gets their memberships assigned to them via the assignMembership method
- The updateMemberInfo() method is called when gym staff are obligated to update a member's information

MembershipPlan Class

The purpose of this class is to present the details of a membership plan (duration, type, fee)

Attributes

- planName: String - eg student plan
- durationMonths: int - represents the length of the plan in months
- Fee: double - price

Methods

- getPlanDetails() - returns the plan name, the duration, and the price

Interactions

- GymManagementSystem.createMembershipPlan() is used to create plans

Membership Class

The start and end dates of each membership are tracked using the membership class. It is also responsible for calculating membership fees, and checking membership status

Attributes

- startDate: Date - membership start date
- endDate: Date - the end date of the membership
- Active: boolean - false if the membership is expired and true if it's active

Methods

- calculateFee() - returns fee based on the plan that was chosen
- checkStatus() - checks if the membership is active or expired

Interactions

- It is created when GymManagementSystem.assignMembership() is called
- It links a member to a membership plan through the assignment in the system

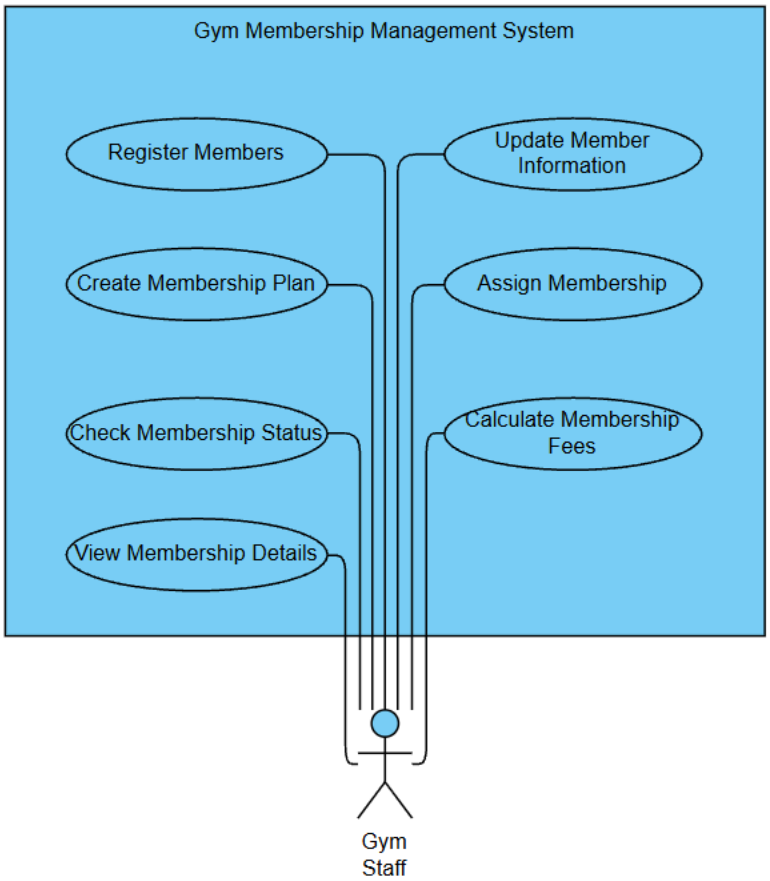
GymManagementSystem Class

The purpose of this class is to be the central class that is managing all the main operations of the system. It coordinates the creation of members, memberships and their assignments, and also displays the membership details.

Methods

- registerMember(name: String, contactInfo: String): member - this is what is used to return the member object and create a new member
- createMembershipPlan(name: String, duration: int, fee: double): MembershipPlan - this is used to create a new membership plan
- assignMembership(member: Member, plan: MembershipPlan): Membership - this is used to assign a membership to a member
- displayMembershipDetails(memberID: int): void - displays the personal and membership information of a member

3.2 Use Case Diagram



There is one actor in the use case diagram for this system and it is the gym staff.
Use cases

- Register members
- Update member information
- Create membership plan
- Assign membership
- Calculate membership fees
- Check membership status
- View membership details

The use case diagram is used to show how the actors (gym staff) interact with the system. The gym staff are the only actors because of how the system functions can all be performed by the gym staff. In this system there are no separate logins for customers or admins because it is a desktop system.

Register members: staff can register and add new members with a name and contact information.
Update member information: staff can edit the details of already existing members.
Create membership plan: staff can add and define new plans with different fees and durations.
Assign memberships: staff can assign memberships to a registered member.
Calculate membership fees: the staff can calculate the fees of a certain membership depending on the membership type and its duration.
Check membership status: this is how the staff see if a member's membership is active or not.
View membership details: staff can see the details of the member and the membership they have assigned to them.

4. OOP Principles

This was a system that was designed using object oriented programming principles because they help improve maintainability and scalability.

4.1 Encapsulation

This is used to maintain and ensure controlled access to data in certain objects. Each class represents an entity that stores related attributes and methods together. An example of this is the member class. The member class contains memberID, name, email, and phone number. It also contains methods that allow these values to be accessed when needed.

4.2 Abstraction

This is how the complex internal operations of the system are hidden from the user. The GymManagementSystem class has methods that are simplified such as registering members and creating plans. These high level operations are what the users interact with without needing to understand any of the internal details.

4.3 Singleton Design Pattern

This pattern ensures that only one instance of the system exists during the execution. Using this design prevents confusion because of how the data is all managed from a single central point.

5. Program Flow

The application starts and the GUI displays the main menu. This main menu consists of several options. These options allow gym staff members to interact with the system using buttons and perform operations.

The typical system flow:

1. The user selects an option from the main menu
2. Then the system asks the user to input relevant information
3. The GymManagementSystem processes that request
4. Objects like Member, Membership, MembershipPlan are created
5. Then the result is displayed

This is an example of how this system is organized and structured and how each class does its specific role.

6. Testing

1. Main Menu

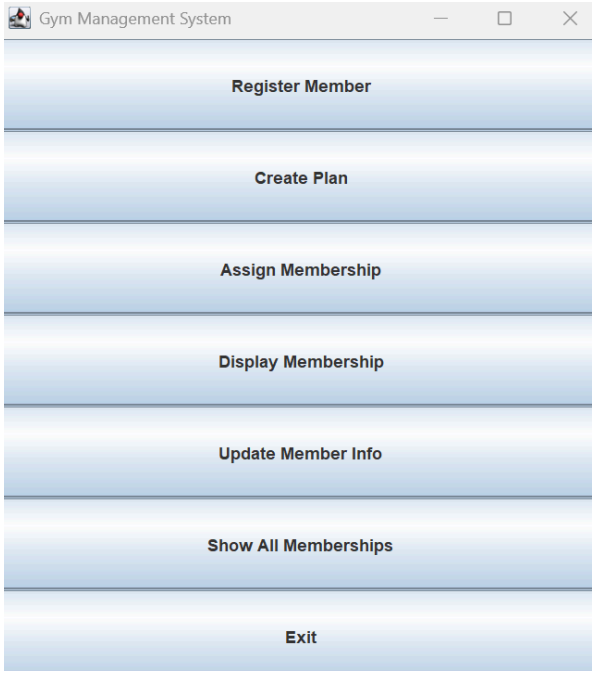
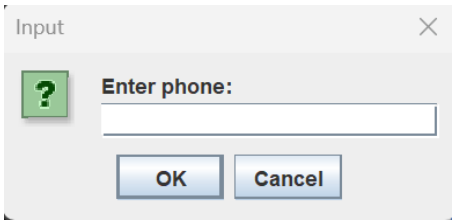
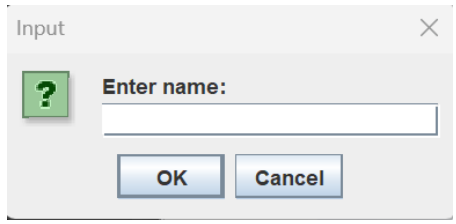


Figure 1: shows the main menu of the system.

2. Customer registration





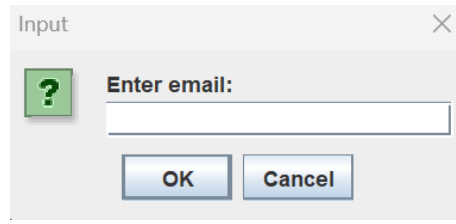
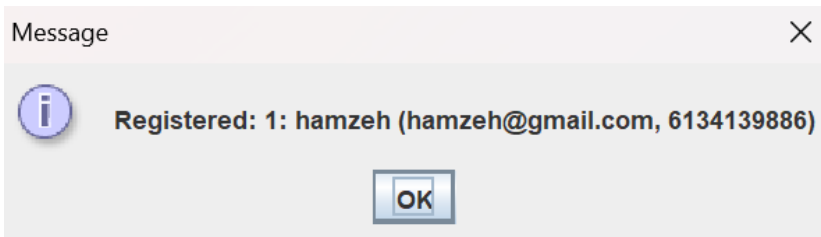


Figure 2, 3, and 4: show the steps for customer registration.



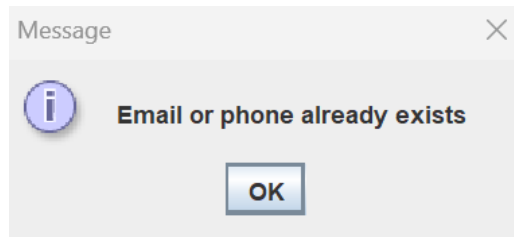
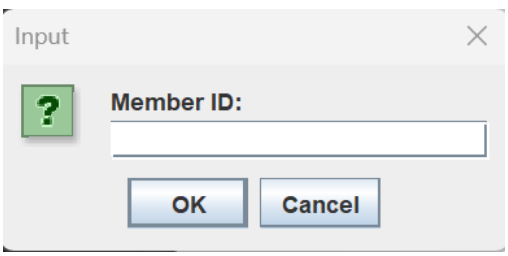


Figure 5 and 6: shows the message output when a member is registered successfully or unsuccessfully

3. Assigning a membership



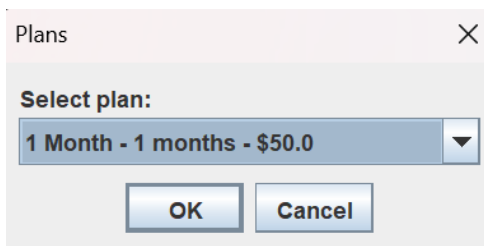


Figure 7 and 8: show the process of assigning a membership.

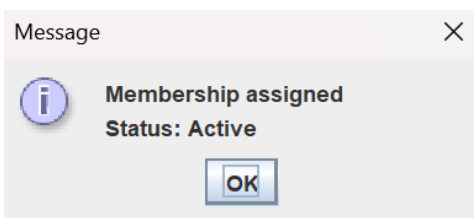
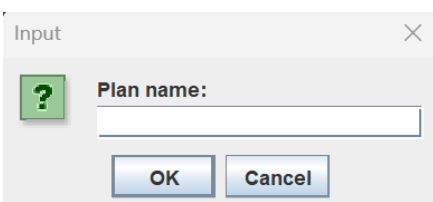
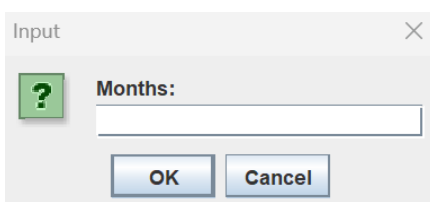


Figure 9: shows the message output when a membership is assigned

4. Creating a plan





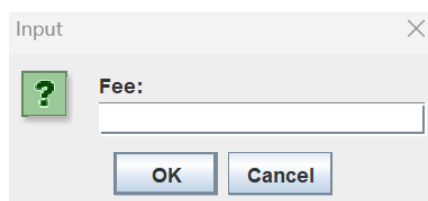


Figure 10, 11 , and 12: show the process of creating a new membership plan.

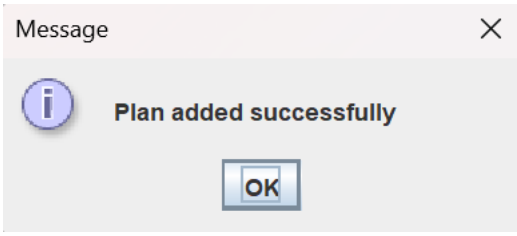


Figure 13: shows the message output for when a new plan is created.

5. Display membership

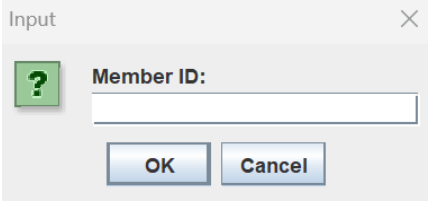


Figure 14: shows the process of displaying membership plans.

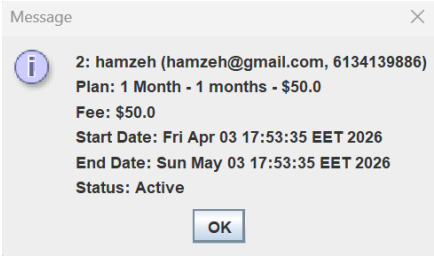
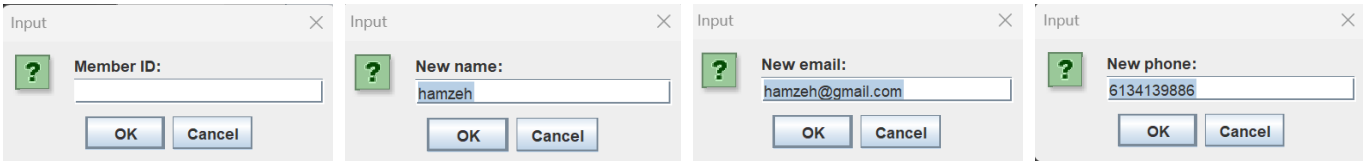


Figure 15: shows the message output when a membership is displayed

6. Updating member information



Figures 16, 17, 18, and 19: show the process of updating a member's information.

7. Showing All Memberships

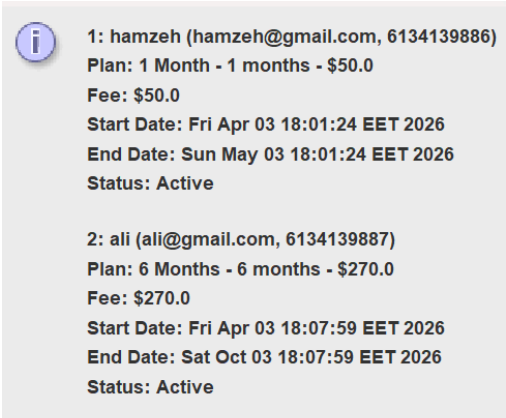


Figure 20: shows the output after clicking the “show all memberships” button

These tests were conducted to make sure that the system runs smoothly and doesn't have any errors in the logic. All tests ran successfully and the system behaved as needed.

7. Conclusion

This system provides a structured and efficient way to handle and manage information about gym members and memberships. Also, it models real world entities such as members and membership plans as well.

This system serves its purpose by successfully allowing gym staff to register members, create and manage membership plans, assign memberships to its members, and update member information.